

ClimateReady Africa making of abstract

Yannick Aurelien Wabo | 9218017 | DLMCSPSE01

Phase 3 | 26 September 2026 | Submission deadline 26 September 2026

Content and concept

ClimateReady Africa is a public academic demonstration supporting early mitigation-project preparation. It asks which evidence is available for a selected carbon-market pathway and what preparation actions should follow. Project developers and consultants are the primary audience. A visitor selects or creates a fictional project, rates applicable criteria, records evidence notes and receives a readiness score, assessment completeness, critical gaps, prioritised actions and a downloadable text report. Four fictional African projects illustrate contrasting sectors and pathway choices.

The rubric contains five pillars and up to fourteen criteria. Pathway, intended use and sector determine applicability. Weighted ratings distinguish missing, partial and documented preparation. A critical gate prevents a high numerical score from concealing unresolved essential items. The weights and thresholds are academic design choices; the system does not establish eligibility, authorisation, registration, certification or verified mitigation. Evidence notes are self-reported and their contents are not independently validated.

Development method and technical approach

Reading and coding began approximately one month before the Phase 1 submission on 3 September 2026. Phase 2 was submitted on 17 September following specification refinement and code development. Finalisation runs to 26 September. A solo iterative Agile approach connects requirements, implementation, checks and documentation. Assessor feedback led to a complete work-package catalogue and schedule, measurable quality targets, explicit UML diagram types and a single maintained specification. Historical package dates and effort are reconstructed estimates rather than timesheets.

The implementation uses React and TypeScript, Next-compatible routes through Vinext/Vite, a Cloudflare Worker and D1. Pure domain functions calculate applicability, scores, status, actions and report text. Zod validates project and assessment data. In version 1.0.1, D1 supplies fictional templates and public creation/update endpoints reject requests. Editable copies and new projects stay in the visitor's browser. Web Locks and project revisions protect local edits across tabs; reset restores only the current browser workspace. Clearing browser data removes local work. Docker Compose, locked dependencies and GitHub Actions provide installation and release procedures.

Verification and release status

Local verification on 25 September passed 15 unit tests, lint and TypeScript checks. Seven existing tests cover rubric behaviour, output and profile validation; eight added cases cover template-copy isolation, persistence, scope resets, stale updates, evidence validation, reset, storage failure and corrupt data. Four HTTP checks through the local Worker-compatible runtime confirm that fictional examples are returned, anonymous server writes receive 403 and shared content remains unchanged. The tests and logs are included with the revised source.

An earlier review of version 1.0.0 found that visitors could overwrite shared examples. Version 1.0.1 addresses that defect with read-only server data and local visitor copies. This illustrates why a successful unit suite or compilation cannot alone establish a safe public workflow. The local workspace tests use independent storage test doubles; a hosted browser walkthrough remains necessary. The review reused a previously installed dependency tree after an offline clean install encountered an uncached package. Docker execution, hosted performance, selected accessibility checks, maintenance timing and rollback remain unverified.

Lessons learned and critical evaluation

The strongest process lesson is to distinguish requirements, design decisions and measured results. Early scope included accounts, administration and a different backend. The final demonstration uses one TypeScript deployment, code-controlled rules, local visitor workspaces and text reports. Recording these changes and updating the architecture avoided implying that proposed features were implemented. Stable work-package and requirement identifiers support traceability, while effort estimates remain clearly separated from actual submission milestones.

The contribution is an explainable preparation workflow connecting conditional criteria, evidence maturity and actions. Its limitations include uncalibrated weights, synthetic examples, a solo author and no independent usability or domain-validation study. Claims of faster preparation or stakeholder acceptance therefore remain hypotheses. Frameworks, tools, domain sources and third-party dependencies are identified in the consolidated report and source documentation; applicable licences are retained.

The revised source and documentation are prepared for final submission. At the evidence cut-off, the public GitHub repository and hosted application have not been verified. The remaining actions are publication, hosted checks, entry of actual URLs, source-archive equivalence and university submission formalities. The delivered package must be completed with those records and the exact earlier submission files before the final source freeze.